electronics

MDPI

Article

Machine-Learning-Based Multi-Corner Timing Prediction for Faster Timing Closure

Zhenyu Zhao , Shuzheng Zhang , Guoqiang Liu *, Chaochao Feng, Tianhao Yang, Ao Han and Lei Wang

College of Computer Science and Technology, National University of Defense Technology, Changsha 410000, China; zyzhao@nudt.edu.cn (Z.Z.); zhangshuzheng17@nudt.edu.cn (S.Z.); fengchaochao@nudt.edu.cn (C.F.); yangtianhao_2020@nudt.edu.cn (T.Y.); hanao19@nudt.edu.cn (A.H.); leiwang@nudt.edu.cn (L.W.)

* Correspondence: liuguoqiang14@nudt.edu.cn

Abstract: For the purpose of fixing timing violations, static timing analysis (STA) of full-corners is repeatedly executed, which is time-consuming. Given a timing path, timing results at some corners ("dominant corners") are utilized to predict timing at other corners ("non-dominant corners"), which can greatly shorten the runtime of STA. However, the huge number of combinations of the dominant corners and the wide difference in prediction accuracy make it difficult to apply multi-corner timing prediction to chip industrial design. In this paper, we propose a dominant corner selection strategy to quickly determine the dominant corner combination with high prediction accuracy, along with which a new multi-corner timing prediction process is established to speed up STA. Experimental results show that our method can not only effectively accelerate STA, but also ensure the high prediction accuracy of the prediction timing. On the public ITC'99 benchmark, the prediction accuracy of the dominant corner combination selected by the proposed method is up to 98.2%, which is an improvement of 15% compared to the state-of-the-art method. For industrial application, we apply our method by using timing results on only 2 dominant corners to predict the other 12 non-dominant corners, which accelerates the runtime of the timing closure process by more than 2×.

Keywords: machine learning; chip physical design; STA; multi-corner; timing closure

check for updates

Citation: Zhao, Z.; Zhang, S.; Liu, G.; Feng, C.; Yang, T.; Han, A.; Wang, L. Machine-Learning-Based Multi-Corner Timing Prediction for Faster Timing Closure. *Electronics* **2022**, *11*, 1571. https://doi.org/10.3390/electronics11101571

Academic Editor: Athanasios Kakarountas

Received: 4 April 2022
Accepted: 11 May 2022
Published: 13 May 2022

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Static timing analysis (STA) [1] is one of the most important techniques available to validate the timing of a chip circuit. It looks for timing violations by checking the timing results of all timing paths. The timing of a path is affected by many factors, including process, voltage, temperature, and parasitic interconnect. The combination of these factors is referred to as a corner. For the purpose of covering all possible timing violations, STA must be performed at all corners, which is time-consuming in achieving timing closure. Given a timing path, the timing results at different corners are closely related [2]. Consequently, the timing results at some corners (described as "dominant") can be used to predict those at the remaining ones (called "non-dominant"), which significantly reduces the runtime of timing analyses and speeds up timing closure. In a practical design flow, the number of dominant corners can be determined according to the different requirements of accelerating STA (e.g., if the designer want to save more than half of the STA tool's runtime, the number of dominant corners can only account for half of all corners at most).

In order to clearly show the prediction accuracy difference of different dominant corner combinations, the size of dominant corner combinations must be consistent. We use Figure 1 to illustrate the wide accuracy divergence of different combinations: the experimental data originate from an industrial design with 14 hold corners (suggested by the foundry; the details are shown in Table 1) in a 16 nm process. We evaluate the prediction accuracy of slack (the difference between the required time and arrival time of data) with a simple linear multivariate regression as the prediction engine. The number

of dominant corners is set to 7. According to the combination formula, the number of dominant corner combinations can be calculated to be as high as 3432 (C_{14}^{7}). Each point represents one combination. Notably, the upper and lower limits of prediction accuracy (percentage of all result when error <10 ps) are 68.9% and 97.6%, respectively. Thus, the maximum divergence is up to 28.7%. The numerous combinations and the massive accuracy divergence make the choice of dominant corners extremely important and difficult.

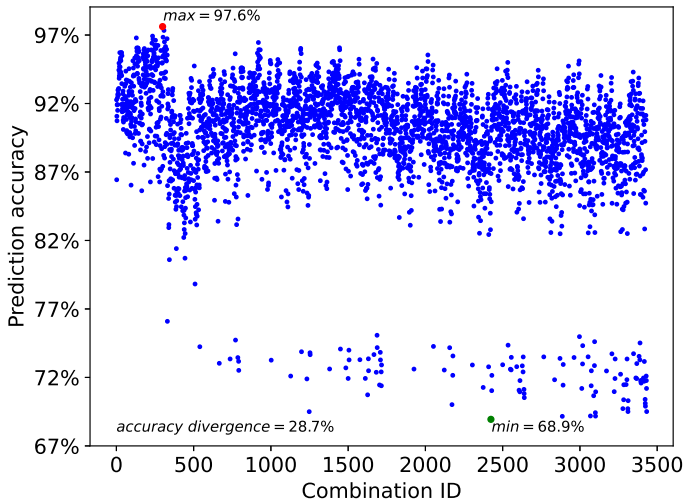


Figure 1. Prediction accuracy distribution for *art1*. Each point represents a dominant corner combination. The abscissa value is the ID of the combination, and the ordinate value is the prediction accuracy corresponding to the combination. The maximum accuracy, minimum accuracy, and the difference between the two are all marked.

Table 1. Parameter Value of Corner.

Corner ID	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Process	*ff*	*ff*	*ff*	*ff*	*ff*	*ff*	*ff*
Voltage (V)	0.88	0.88	0.88	0.88	0.88	0.88	0.88
Temperature (°C)	0	0	0	0	125	125	125
Library-Corner	*bc*	*bc*	*bc*	*bc*	*ml*	*ml*	*ml*
RC-Corner	*cbest*	*cworst*	*rcbest*	*rcworst*	*cbest*	*cworst*	*rcbest*
Corner ID	**8**	**9**	**10**	**11**	**12**	**13**	**14**
Process	*ff*	*tt*	*tt*	*ss*	*ss*	*ss*	*ss*
Voltage (V)	0.88	0.8	0.8	0.72	0.72	0.72	0.72
Temperature (°C)	125	85	85	0	0	125	125
Library-Corner	*ml*	*tc*	*tc*	*wcz*	*wcz*	*wc*	*wc*
RC-Corner	*rcworst*	*cworst*	*rcworst*	*cworst*	*rcworst*	*cworst*	*rcworst*

This work introduces a machine learning approach to multi-corner timing prediction. A dominant corner selection strategy is implemented to look for dominant corner combinations with a high prediction accuracy. Then, the timing closure process is accelerated as a "plug-in" for STA tools. The contributions of this work include the following:

- It is shown for the first time that different dominant corner combinations have different prediction accuracies, and that the divergence is particularly huge.
- We put forward an efficient dominant corner selection strategy using a non-linear model which can be used to quickly and generally select the combination of dominant corners that meets the requirements of STA acceleration and prediction accuracy.
- We propose an application flow for multi-corner timing prediction based on our proposed dominant corner selection strategy. A method of incremental re-training

which can handle outliers or include new data in a timely manner is integrated into the flow to improve the efficiency of the training process .
- We apply our method to industrial design and prove that the machine learning-based method can achieve a faster timing closure.

The rest of this paper is organized as follows. Section 2 introduces related work. Section 3 explains the meaning of a corner, and further expounds the necessity of our method and data construction method. Section 4 describes our work in detail. Section 5 reports experiments and results. Section 6 concludes the paper.

2. Related Work

Accelerating STA for faster timing closure is a valuable part of research in chip physical design. A full timing run for a few corners and partial timing runs for others are performed, and they are combined them to find the worst-case hold slacks. Silva et al. propose an efficient automated methodology in [3] for computing the worst-delay process corners. In [4], Onaissi et al. exploit a linear-time approach for STA which covers all process corners in a single pass. Michael et al. describe an efficient statistical timing analysis algorithm in [5] that can handle arbitrary (spatial and structural) causes of delay correlation. Vishal et al. present a general STA framework in [6] that captures spatial correlations between gate delays. The framework reduces the computational complexity introduced due to polynomial modeling during STA, thus accelerating the STA scheme. Jing-Jia et al. proposed a unified Multi-Corner Multi-Mode STA engine in [7] that can efficiently compute the worst-case delay of the process corners in various very large circuits. Onaissi et al. present an alternative method for performing fast and accurate hold timing analysis in [8] which covers all corners. However, this work also requires timing analysis of clock network on the other corners.

In recent years, machine learning methods have also been applied in STA acceleration. Bian et al. researched an NBIT aging prediction in [9] to eliminate the aging effect on timing and achieved a maximum absolute error of 3.42%. In [10], Kahng et al. use an ML predictor to predict signal integrity (SI) mode timing from the timing reports of non-SI mode analysis. Han et al. have developed a machine learning-based tool, GTX [11], to correct divergence between two STA tools. A learning approach is proposed in [12] to estimate wire slew and delay, reducing the number of invocations of the signoff STA tool. Chan et al. used machine learning algorithms to evaluate the timing slacks of embedded SRAM [13]. Guo et al. propose an efficient implementation for accelerating STA on a GPU and implemented their algorithms on top of OpenTimer, which achieve up to 3.69× speed-up on a large design of 1.6 M gates and 1.6 M nets using one GPU [14].

Most work has focussed on the "inside" improvement of STA tools. This means that they accelerate the speed of timing calculation processes inside STA tools by improving algorithms. In contrast, we treat the STA tool as a "black box" and seek to accelerate the STA process "outside", which can be regard as a "plug-in" for STA tools. Kahng et al. conducted some related work in [2]. They observe that timing results for a given path at different corners will have strong correlations, and investigate a data-driven approach, based on multivariate linear regression, to predict the timing analysis at unobserved corners from analysis results at observed corners. The result of the timing prediction at unobserved corners and the relative root mean squared error is less than 0.5%. This is the first work that applied machine learning to accelerate the timing closure process by using correlation between different corners, but it still contains some problems to be solved : (1) It is possible to learn more flexible statistical models rather than linearity models; (2) the model needs to handle outliers or include new data in a timely manner; (3) the optimal combination of strategies for STA acceleration it remains to be found. In this work, we address these problems and verify the problem in an actual industrial application to show the potential application in physical IC design.

3. Preliminary

3.1. Definition of Corner

A timing corner is composed of a *Library-Corner* and an *RC-Corner*. The design timing is affected by process (P), voltage (V), and temperature (T), which are derived from the timing library provided by the semiconductor foundry. It is thus defined as the *Library-Corner*, also known as PVT condition [15]. When the chip achieves timing closure in these corners, the function and performance of it can be guaranteed. With the emergence of nanometer technology, the sensitivity to parasitic interconnect can no longer be ignored [16] and the impact on timing is considered. In parasitic interconnect, resistance and capacitance are the most important variables, and thus, the influence of parasitic interconnect on timing is defined as the *RC-Corner*. In this paper, we perform research based on the timing results from 14 corners, which are marked with Corner ID (1–14) along with the specific parameter values shown in Table 1.

3.2. Exploration of Dominant Corner Space

In order to further illustrate the problem of multi-corner timing prediction, we extended the experiment mentioned in the introduction: we set the number of dominant corners to n (the value of n increases from 1 to 6 in steps of 1), and the prediction accuracy of each dominant corner combination is evaluated to show the ubiquitous divergence. Figure 2 shows the evaluation results.

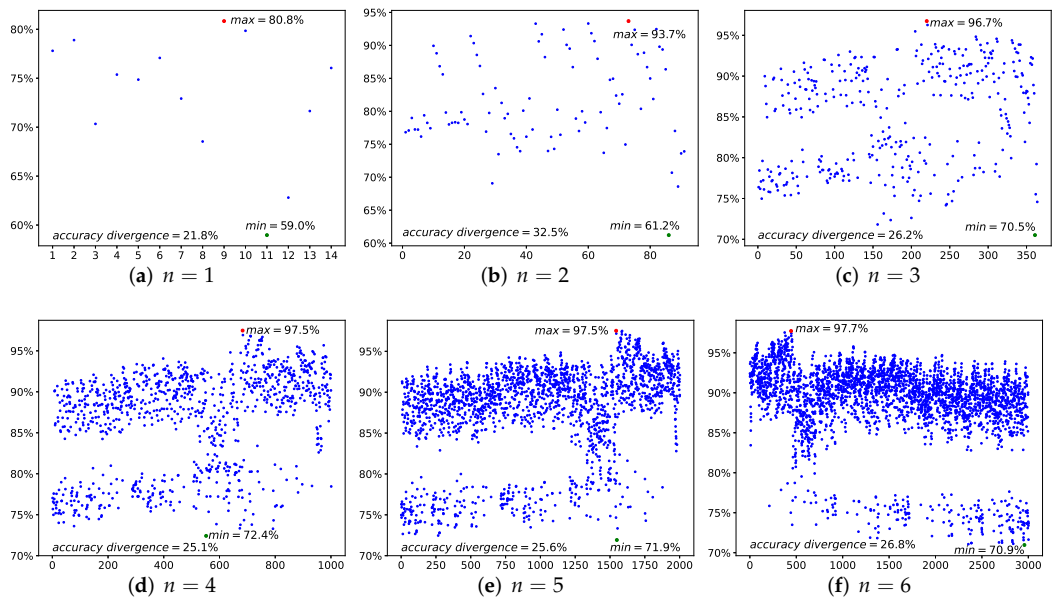


Figure 2. Extended Experiment. Subfigures (**a**–**f**) show all dominant corner combinations when the number of dominant corners varies from 1 to 6. n refers to the number of dominant corners, and each blue dot represents a dominant corner combination.

Based on the experimental results, we summarize three challenges in the application of multi-corner timing prediction. **Challenge 1:** the number of dominant corner combinations increases explosively with the increase in the number of dominant corners. When n changes from 1 to 6, the number of combinations increases from tens to thousands. **Challenge 2:** The accuracy divergence is tremendous, no matter what the value of n is. When $n = 2$, the accuracy divergence is as high as 32.5%. The average divergence of all cases reaches 26.8%. **Challenge 3:** The acceleration effect of STA and the prediction accuracy of inference are mutually restricted. The larger the value of n is, the more dominant corners are, and the longer a runtime of the timing analysis is required. At the same time, the increase in training data also improves the prediction accuracy of the timing prediction model. In

summary, there are still big challenges for integrating multi-corner timing prediction into the industrial design flow.

3.3. Data Construction

The timing-related data are extracted from the timing report. We design an automatic data generation flow to acquire timing results.

As shown in Figure 3, the flow includes two steps: extraction of timing path and extraction of timing result. In step 1, we use a commercial STA tool to analyze timing results of $\#num$ worst timing paths at N corners, and then unify all the paths to remove duplicate paths. After this, we acquire a path set as $Path_{union}$, which contains k paths. In step 2, we still employ the same STA tool to obtain the timing results of each path in $Path_{union}$ at N corners. Subsequently, timing data are organized into a two-dimensional matrix.

$$\text{Step 1:}\quad \text{Design} \Rightarrow \begin{Bmatrix} Corner_1 \\ \dots \\ Corner_N \end{Bmatrix} \overset{{}^*\#num \text{ worst paths}}{\Downarrow} \Rightarrow \text{STA Tool} \underset{Extract}{\overset{Path}{\Rightarrow}} \begin{Bmatrix} {}^*\boldsymbol{Path_1}\{N\times 1\} \\ \dots \\ \boldsymbol{Path_{num}}\{N\times 1\} \end{Bmatrix} \bigcup \Rightarrow {}^*\boldsymbol{Path_{union}} = \boldsymbol{Path_{1\sim k}}\{N\times 1\}$$

$$\boldsymbol{Secondary\ Timing\ Analysis}$$

$$\text{Step 2:}\quad \text{Design} \Rightarrow \begin{Bmatrix} Corner_1 \\ \dots \\ Corner_N \end{Bmatrix} \Rightarrow \text{STA Tool} \underset{Extract}{\overset{Timing}{\Rightarrow}} \boldsymbol{Timing} = \begin{bmatrix} & Corner_1 & \dots & Corner_N \\ path_1 & timing_{1,1} & \dots & timing_{1,N} \\ \dots & \dots & \dots & \dots \\ path_k & timing_{k,1} & \dots & timing_{k,N} \end{bmatrix}$$

$*$ $\#num$ worst paths: Set the number of timing paths to $\#num$

$*$ $\boldsymbol{Path_i}\{\boldsymbol{N}\times\boldsymbol{1}\}$: It represents the timing results corresponding to N corners of ith timing path $(i=1,\dots,\#num)$

$*$ $\boldsymbol{Path_{union}}$: A collection of timing paths without duplication

Figure 3. Data generation flow.

3.4. Machine Learning Models

Different models have their own strengths and weaknesses. In this work, we researched the following models to find the model that we need, as shown in Chapter 5.1.

Ridge: This is a linear regression model [17]. $f(x) = x^T w + b$ is the formulaic representation, and w is regression coefficient in it. During training, in order to prevent overfitting, L2 regularization is incorporated into Ridge's loss function J ($J = ||f(x) - y||_2^2 + \lambda||w||_2^2$). Ridge's greatest advantages are its simplicity and explainability. However, it may be too simple to solve complex problems.

MLP: Multilayer Perceptron (MLP) is an artificial neural network [18]. The data from the previous layer are transformed by many neurons with a non-linear activation function in the present layer. Strong adaptability is the main advantage of MLP, as it can quickly adapt to new problems. Meanwhile, the large amount of training data needed and the lack of interpretability make it difficult to apply.

Random Forest: This establishes multiple decision trees and fuses them to achieve a more accurate and stable model [19]. In the process of prediction, Random Forest counts the prediction results of all trees, and then selects the final result by voting. The advantage is to avoid overfitting as much as possible by considering the results of trees. However, too many decision trees will slow down the model.

4. Timing Prediction Method and Experiment Preparation

4.1. Machine-Learning-Based Multi-Corner Timing Prediction Method

Our aim is to integrate machine-learning-based multi-corner timing prediction into the chip physical design flow, so as to greatly accelerate STA. Our method is mainly composed of two parts,

including a dominant corner selection strategy (iterative increase strategy) and an application flow of multi-corner timing prediction. The former is used to quickly obtain the dominant corner combination that meets the requirements of STA acceleration and prediction accuracy. The latter is used to guide the usage of multi-corner timing prediction in industrial design.

4.1.1. Iterative Increase Strategy for Dominant Corner Selection

Inspired by feature selection in [20], a machine learning model is embedded in our selection strategy to find a new dominant corner iteratively. Assuming that the maximum number of dominant corners is n, our strategy requires n iterations to get a performance assessment form (*PAF*), which is a list that guides the determination of dominant corner combinations. Figure 4 shows the flow of our selection strategy in detail.

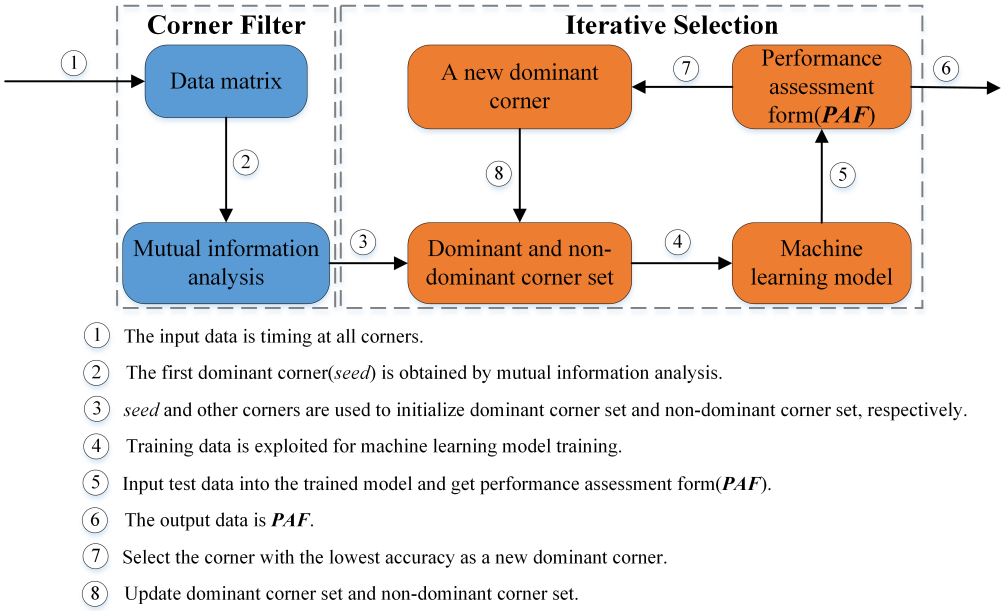


Figure 4. Iterative increase strategy.

Corner Filter: The purpose of this step is to obtain the first dominant corner, which is named *seed*. Timing results at all corners are input data of our selection strategy, and we reorganize them into a two-dimensional matrix. As mentioned earlier, the timing results of the same path at different corners have strong correlations. Therefore, we exploit a mutual information analysis method, the pearson correlation coefficient [21], to determine *seed*, which is the most relevant corner. The process of acquiring *seed* is divided into two stages. Stage 1: we use Equation (1) to calculate the correlation coefficient value r_{xy} between x and y. x and y are vectors of the form $\begin{bmatrix} x_1 & \dots & x_k \end{bmatrix}^{\mathrm{T}}$ and $\begin{bmatrix} y_1 & \dots & y_k \end{bmatrix}^{\mathrm{T}}$, representing the timing of paths at different corners. k is the number of paths and i is the specific timing path. Stage 2: Equation (2) is invoked to sum the correlation coefficient value of each corner. If R_x is the largest, then corner x is *seed*.

$$r_{xy} = \frac{k\sum x_i y_i - \sum x_i \sum y_i}{\sqrt{k\sum x_i^2 - (\sum x_i)^2}\sqrt{k\sum y_i^2 - (\sum y_i^2)}} \tag{1}$$

$$R_x = \sum_{y \neq x} r_{xy} \tag{2}$$

Iterative Selection: Once *seed* is obtained, the selection algorithm can be started. *Seed* and the remaining corners are used to initialize the dominant corner set and non-dominant corner set separately. Next, dominant corners and non-dominant corners are utilized as features and attributes to train the machine learning model. To evaluate the prediction accuracy of the dominant corner combination, we run the trained model with the test model.

The generated evaluation data are added to *PAF* and the output. In the meantime, the non-dominant corner with the lowest prediction accuracy is selected as the new dominant corner. The last action is to update both the dominant corner set and the non-dominant corner set. At this point, a complete selection process is finished. Each time we run the algorithm, we can filter out a dominant corner combination with the highest prediction accuracy and a new dominant corner for the next iteration. Algorithm 1 describes the iterative selection process in detail.

Algorithm 1 Dominant corner selection algorithm

Input: The initial corner, *seed*; The set of all corners, α; Timing results at all corners in matrix form, T; The maximum number of dominant corners, *max*; A machine learning model, *model*;
Output: *PAF*, *score*;
1: Creating empty dominant corner set β and non-dominant corner set γ;
2: Initializing β and γ:
$\beta \leftarrow seed, \gamma \leftarrow \alpha - \beta$;
3: Partitioning training data T_{train} and predicting data $T_{predict}$:
$T \leftarrow T_{train} + T_{predict}$;
4: **for** $i = 1 \rightarrow max$ **do**
5: $x_{train} \leftarrow T_{train}(\beta), y_{train} \leftarrow T_{train}(\gamma)$;
6: $x_{predict} \leftarrow T_{predict}(\beta), y_{verify} \leftarrow T_{predict}(\gamma)$;
7: Training the machine learning model:
$model(x_{train}, y_{train})$;
8: Using the trained model to predict y_{verify}:
$y_{predict} \leftarrow model\left(x_{predict}\right)$;
9: Evaluation prediction accuracy *acc*, and updating *score*:
$acc \leftarrow \text{Accuracy}\left(y_{verify}, y_{predict}\right), score \leftarrow score + acc$;
10: Selecting the corner corresponding to the lowest accuracy as *temp*, and then updating:
$\beta \leftarrow \beta + temp, \gamma \leftarrow \gamma - temp$;
11: **end for**
12: **return** *score*

After *max* iterations, *PAF* is created. A real example is demonstrated to illustrate the structure and usage of *PAF*. In Table 2, the value of *max* is five and therefore there are 5 dominant corner combinations. The acceleration is affected by the number of dominant corners—the lower the number, the better the acceleration benefit. On the contrary, the prediction accuracy is positively correlated with the number. As a result, *PAF* can help designers to make a trade-off between acceleration effect and prediction accuracy, and to select the most suitable combination. Incidentally, the accuracy evaluation criterion is customizable, which allows our algorithm to be adapted to different goals.

Table 2. An example of *PAF*.

Number	Dominant Corner Combination	Acceleration	Accuracy
1	($corner_8$)	14.0×	79.3%
2	($corner_8$,$corner_3$)	7.0×	85.7%
3	($corner_8$,$corner_3$,$corner_{10}$)	4.7×	90.2%
4	($corner_8$,$corner_3$,$corner_{10}$,$corner_1$)	3.5×	96.7%
5	($corner_8$,$corner_3$,$corner_{10}$,$corner_1$,$corner_{13}$)	2.8×	97.9%

4.1.2. An Application Flow of Multi-Corner Timing Prediction

In this section, we establish an application flow of our multi-corner timing prediction, which is shown in Figure 5.

Training: Timing results of a few paths at N corners are obtained to find the best dominant corner combination. According to the *PAF* generated by our dominant corner selection algorithm, we choose the combination of size n. After that, the timings at n dominant corners and $(N-n)$ non-dominant corners are used as features and labels to train the timing model.

Prediction: The trained timing model can be used to predict the timing of the same design or different designs, as long as these designs use the same chip manufacturing process. Before making the prediction, the timing of large paths at n dominant corners needs to be obtained by the STA tool. Then, the timing at the remaining non-dominant corners can be predicted.

Incremental Re-Training: Generally speaking, the more training data, the better the performance of the model. In order to continuously improve the generalization of the timing prediction model, an incremental training mechanism is integrated into this flow. That is to say, the model is re-trained by the newly obtained timing results at N corners.

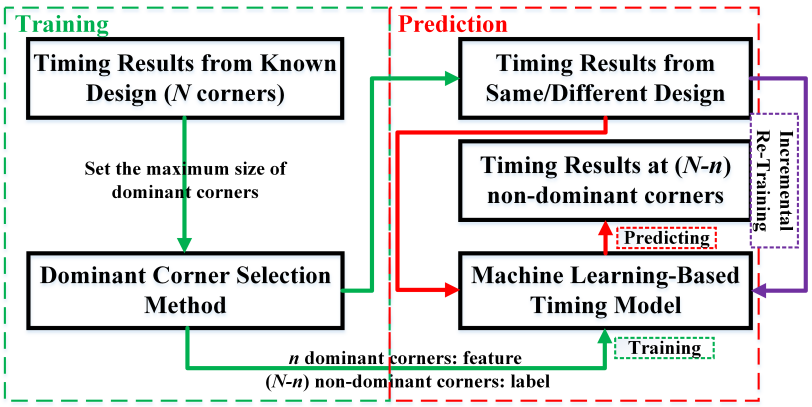


Figure 5. Machine-learning-based timing prediction application flow.

4.2. Experiment Configuration

4.2.1. Information of Designs

Our multi-corner timing prediction is implemented in Python. We evaluate the prediction performance of the three machine learning models mentioned in the previous section, and they are invoked in scikit [22]. We make use of an open source STA tool, OpenTimer [23], to perform timing analysis. We experiment with slack, a type of timing result that is usually used to decide whether the timing path is violative.

Our data are derived from seven designs. *b17*∼*b19* are three public designs, which are from ITC'99 benchmark circuits [24]. *art1* and *art2* are two external interconnection interface circuits designed by ourselves, including a low-speed peripheral interface and a high-speed peripheral interface. *ind1* is a 512 KB L2 Cache with 16 ways associated, while *ind2* is an out-of-order superscalar CPU, which has 13 4-issue pipeline stages. The clock periods of *b17*, *b18*, *b19*, *art1*, and *art2* are all 2ns (the frequencies are 500 MHz), and the clock periods of *ind1* and *ind2* are both 0.43 ns (the frequencies are 2.3 GHz). Detailed information on these designs is given in Table 3.

Table 3. Detailed information on designs.

Design	#Instances	#Paths	Slack Range (ps)
b17	2.12 K	1.5 K	$[-107, 242]$
b18	4.93 K	3.6 K	$[-129, 272]$
b19	9.87 K	8.1 K	$[-165, 303]$
art1	1.02 M	18.9 K	$[-22, 189]$
art2	1.18 M	26.9 K	$[-105, 1271]$
ind1	0.87 M	50.2 K	$[4, 1503]$
ind2	1.58 M	53.7 K	$[0, 348]$

4.2.2. Prediction Performance Metrics

To investigate the prediction performance of our work, we define the evaluation criteria as shown in Table 4. y_{ij} and $\hat{y}_{ij}$ are the actual and predicted values of timing results, respectively. ϵ_{abs}^{ij} is defined as the absolute error between y_{ij} and $\hat{y}_{ij}$.

Table 4. Reporting metrics for prediction accuracy evaluation.

Symbol	Formula	Meaning
y_{ij}	-	Actual timing at ith path, jth corner
$\hat{y}_{ij}$	-	Predicted timing at ith path, jth corner
ϵ_{abs}^{ij}	$\left\|y_{ij} - \hat{y}_{ij}\right\|$	Absolute error between y_{ij} and $\hat{y}_{ij}$
$LESS10$	$\frac{\sum_{i=1}^{k}\sum_{j=1}^{n}\left(\epsilon_{abs}^{ij}<10\right)?1:0}{k \cdot n}$	Percentage of ϵ_{abs}^{ij} less than $10ps$
MAE	$\frac{\sum_{i=1}^{k}\sum_{j=1}^{n}\epsilon_{abs}^{ij}}{k \cdot n}$	Mean absolute error

k is the number of predicted timing paths. n is the number of non-dominant corners.

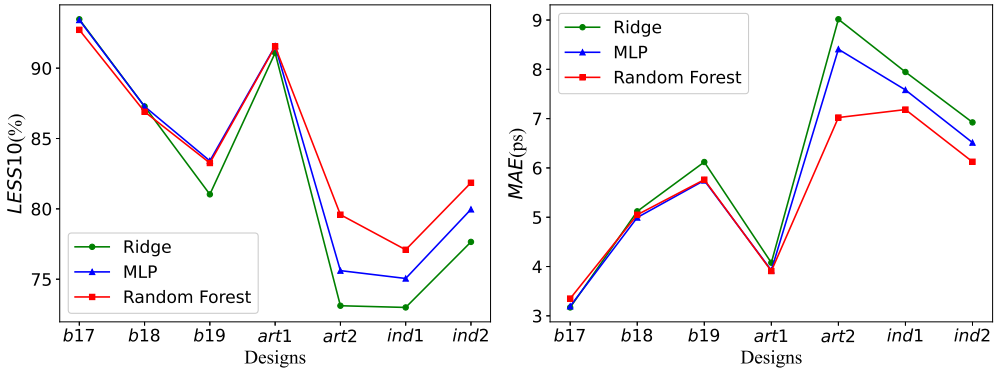
The prediction performance metrics we use are $LESS10$ and MAE. We have learned from IC designers that the predicted timing result is available for timing analysis when the prediction error is less than 10 ps, in today's technology. Note that some works [8] take 10 ps as the design margin and claim that "if the error tolerance represents a very small percentage of clock periods, they can be acceptable in an industrial setting". Furthermore, we compared the $LESS10$ and clock periods: 0.5% for *b17*, *b18*, *b19*, *art1*, and *art2*; and 2.3% for *ind1* and *ind2*. It shows that our prediction error would be acceptable in an industrial design. Accordingly, we put forward the metric $LESS10$, which is the percentage of ϵ_{abs}^{ij} less than 10 ps and can accurately measure the availability of the predicted timing result. MAE is a frequently-used criterion, which reports the mean absolute error of predicted timing results.

5. Experiments and Results

5.1. Experiment 1: Performance Evaluation of Different Models

Ridge, MLP and Random Forest were evaluated in this experiment and the model with the best performance was selected for subsequent experiments. When the number of dominant corners is small, the prediction accuracy of the corresponding combination is lower, making it easier to show the performance of the model. We set the number of dominant corners to 1. To eliminate random error, we investigated all the combinations and obtained the average value of these metrics. In addition, we used GridSearchCV [25] for model parameter tuning.

Figure 6 illustrates our evaluation results. The subfigure on the left is the assessment of $LESS10$. The larger the value, the better the prediction performance of the corresponding model. On the right is the evaluation of MAE. The lower the value, the better the prediction performance. In these two figures, the performance of design *b17*, *b18*, *b19* and *art1* is very similar, and the Random Forest model shows obvious advantages in the last three designs. The experiment shows that, compared with the linear model MLP used in equivalent work [2], the Random Forest model is a more flexible statistical model which can mine more linear and nonlinear information hidden inside the corners. The maximum improvement of $LESS10$ reaches 6.5% and MAE is reduced by 1.99 ps.



(**a**) Evaluation of $LESS10$ (**b**) Evluation of MAE

Figure 6. Performance evaluation of the three models. Subfigure (**a**) shows the evaluation of $LESS10$ of three models on all designs. Subfigure (**b**) shows the evaluation of MAE of three models on all designs.

5.2. Experiment 2: Performance Evaluation of Corner Selection Strategy

In multi-corner timing prediction, the existing strategy that selects the dominant corner used in [2] is greedy deletion. In order to show the ability of our proposed selection strategy, we used these two strategies to determine dominant corner combinations and evaluate the performance of these combinations, respectively. The maximum number of dominant corners was set to 7. We also made use of 5-fold cross validation [26] to reduce error.

The results are shown in Figure 7. As the number of dominant corners increases, $LESS10$ of the two strategies gradually increases and approaches 100%, while MAE continues to decrease. However, it can be clearly seen that the performance of our strategy is much better than that of the existing strategy. In (e) and (f), the performance superiority of our strategy is most obvious—when the number of dominant corners is equal to 2, the acceleration of STA is 7×. $LESS10$ and MAE of our strategy are 98.2% and 2.18 ps, respectively. The corresponding values of the constructive strategy are 83.2% and 5.74 ps. Consequently, the maximum improvement of $LESS10$ is 15.0% and the maximum reduction of MAE is 3.55 ps. Experimental results show that, compared with the existing strategy in a similar work [2], our strategy can select the dominant corner combination with better prediction performance.

5.3. Experiment 3: Further Comparison and Analysis of Prediction Performance

In order to demonstrate the performance of our iterative increase strategy in depth, we conducted an extended experiment on the basis of Experiment 2: we evaluated the prediction performance of all dominant corner combinations with number from 1 to 7, and the data come from design *ind1*.

Table 5 shows the result. In the evaluation of $LESS10$, the performance of our proposed strategy is very close to or even reaches the best level (the maximum difference between max and $\boldsymbol{I}$ is 1.8% when $Nums = 2$), and the greedy deletion strategy is far worse (the minimum difference between max and $\boldsymbol{D}$ is 2.5% when $Nums = 1$). There is a similar error distribution in the MAE evaluation. The experimental data once again show that our strategy can accurately select the combination that is closest to or equal to the upper limit of predicted performance from the massive dominant corner combinations.

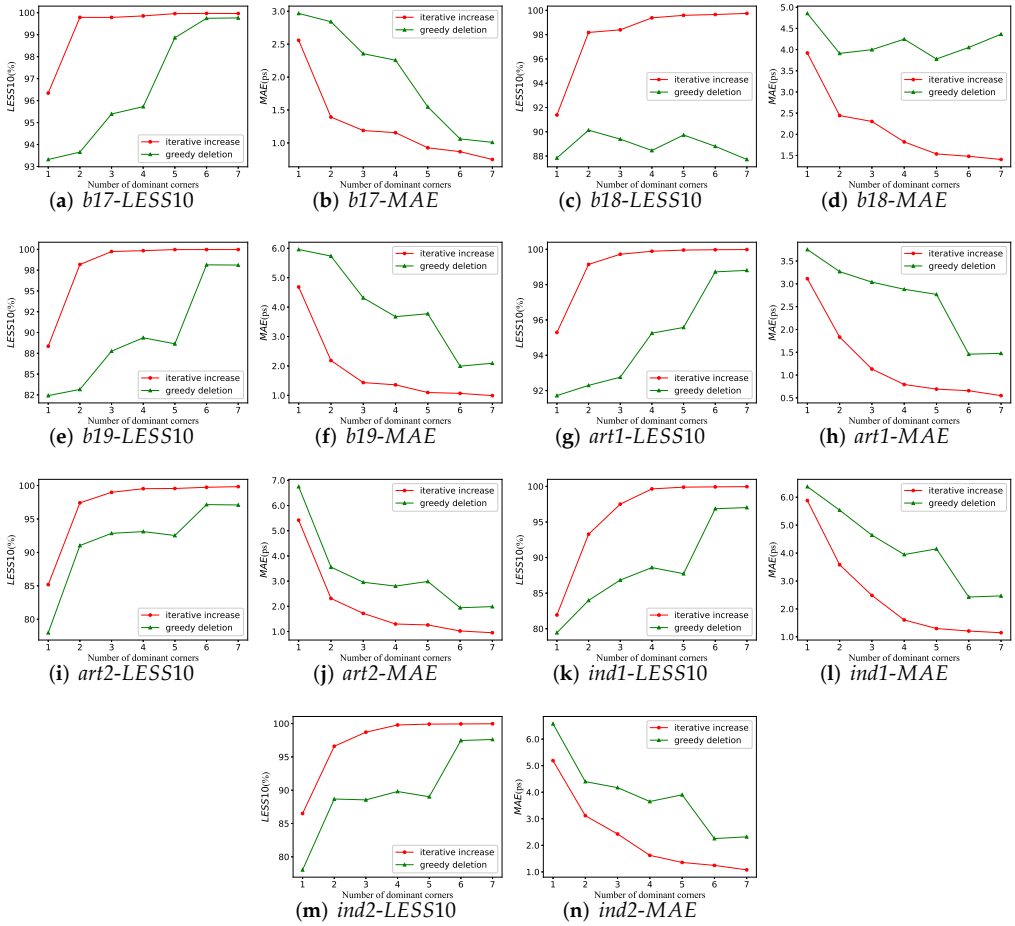


Figure 7. Performance evaluation of the two selection algorithms: Subfigures (**a**–**n**) show the evaluation of *LESS*10 and *MAE* of two strategies on all designs. Plots of our iterative increase strategy (red) and the existing greedy deletion strategy (green).

Table 5. Performance comparison analysis.

Nums	*Combs*	*LESS*10				*MAE* (ps)			
		Max	*Min*	*I*	*D*	*Max*	*Min*	*I*	*D*
1	14	81.9%	70.5%	81.9%	79.4%	8.93	5.88	5.88	6.38
2	91	94.6%	74.5%	93.3%	84.0%	7.51	3.32	3.58	5.54
3	364	99.3%	82.9%	97.5%	86.8%	5.89	1.98	2.48	4.64
4	1001	99.9%	83.1%	99.6%	88.6%	5.56	1.49	1.61	3.95
5	2002	99.9%	87.4%	99.9%	87.7%	4.22	1.20	1.30	4.15
6	3003	99.9%	86.1%	99.9%	96.9%	4.49	1.10	1.21	2.43
7	3432	100.0%	84.2%	100.0%	97.0%	5.01	1.02	1.15	2.47

Nums represents the number of dominant corners. ***Combs*** is the size of dominant corner combinations. ***max*** and ***min*** are the maximum and minimum value of prediction performance in all dominant corner combinations when the number of dominant corners is fixed. ***I*** is the performance of our iterative increase strategy. ***D*** is the performance of the existing greedy deletion strategy.

5.4. Experiment 4: Performance Evaluation of Small-Scale Training Data

A more powerful usage of our timing prediction method is to use a few known timings of a design to predict an enormous number of the timing of unknown paths the same design. In this experiment, we used 10% of data for timing model training, and tested on the timing results of the 90% of unseen paths.

Figure 8 shows the experimental results. When the training data only account for 10% of the total data, our multi-corner timing prediction method can show excellent prediction

performance in each design. In most of the designs, $LESS10$ exceeds 95% when the number of dominant corners is 2. For example, in the performance evaluation of design *art2*, when the number is increased from 1 to 2, $LESS10$ increases from 84.1% to 96.1% with an improvement of 12.0%, and MAE decreases from 5.77 ps to 2.80 ps with a diminution of 2.97 ps.

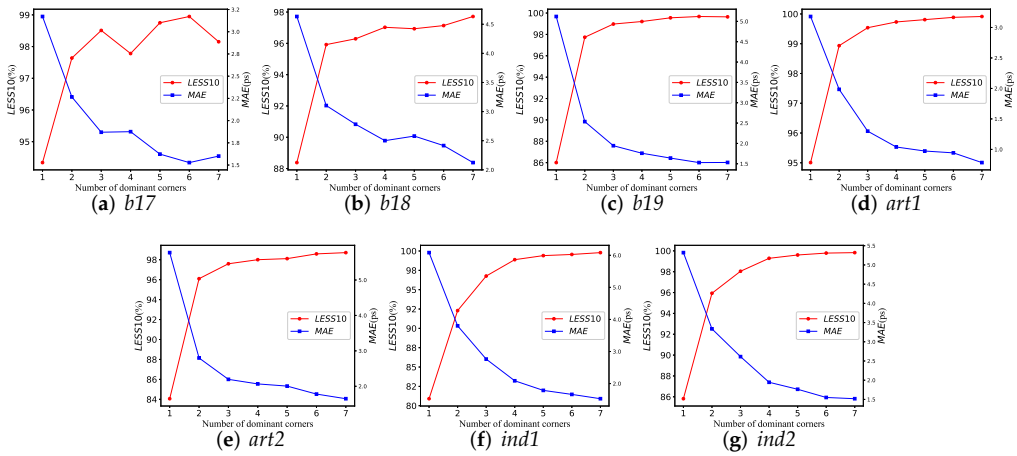


Figure 8. Performance evaluation of small-scale training data. Subfigures (**a**–**g**) show the evaluation of $LESS10$ and MAE on all designs when the number of dominant corners varies from 1 to 7.

5.5. Experiment 5: Performance Evaluation of Incremental Re-Training

Incremental re-training is critical in our timing prediction method, as it ensures that our prediction model can continuously improve the prediction performance by increasing the training data. In this experiment, we predict the slack of *art1* twice. For the first time, only data from *b17* are used as training data; for the second time, data from *art2* are added to re-train the prediction model. The maximum size of dominant corners is still 7.

The evaluation results are shown in Figure 9. According to Table 3 , the slack range of design *b17* and *art1* are relatively close, while the slack range of *art2* completely covers the ranges of the first two designs. Hence, increasing the training data can effectively enhance the prediction performance. When the number is 1, the maximum improvement of $LESS10$ is up to 54.2% and the maximum reduction of MAE is 7.35 ps. The average variations of $LESS10$ and MAE are 31.6% and 5.28 ps, respectively.

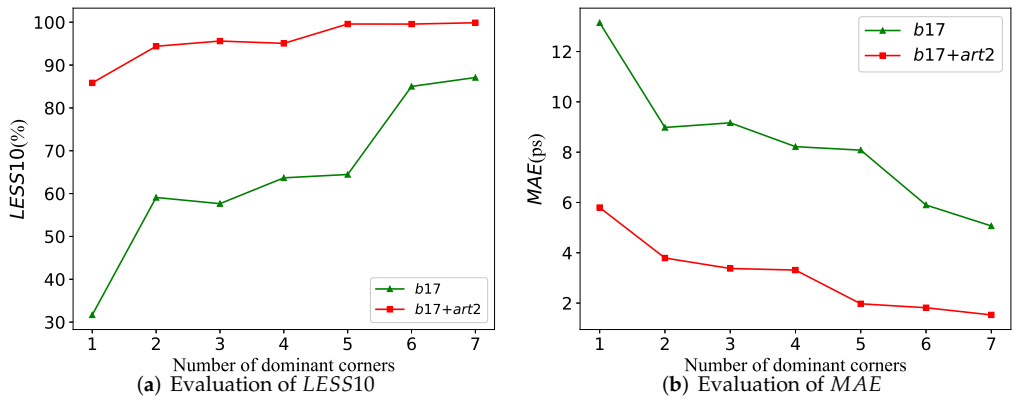


Figure 9. Performance evaluation of incremental re-training. Subfigure (**a**) shows the evaluation of $LESS10$ of our model on *art1*. Subfigure (**b**) shows the evaluation of MAE of our model on *art1*. The green line represents that the training data only comes from *b17*, while the red line is from *b17* and *art2*.

5.6. Experiment 6: Faster Timing Closure in Industrial Application

Considering that most tape-out circuits still use the commercial STA tools today, we applied our method to two industrial designs, *art1* and *art2*, and intuitively show the timing closure acceleration effect of our method. The area of art1 is 1445.84 × 725.952 μm^2 and that of art2 is 1102.05 × 398.88 μm^2. In the commercial timing closure process of chip physical design, an STA tool will be used to calculate the static timing results of all possible timing paths on all 14 corners, and then engineers fix the timing violations. These two steps are repeated until most timing violations are eliminated. Then, the optimized design flows to the signoff stage, which uses an STA tool to meet all the timing constraints.

As shown in Figure 10, the left sub-graph describes the commercial process. According to the above experimental results, our proposed method has enough accuracy to quickly predict the timing result instead of querying the time-consuming STA tool. Thus, the timing results of only 2 corners need to be calculated by the STA tool, and the others are predicted by our machine-learning-based model in each iteration. The right sub-graph of Figure 10 illustrates the machine-learning-based timing closure process. The runtime of STA on one corner is 0.59 h for *art1* and 0.88h for *art2*. The runtime of the machine learning model is only a few seconds. Obviously, we used the model to replace the STA tool on 12 corners, thus accelerating the timing closure process.

The result is shown in Table 6. T_{tool1} is the runtime of the STA tool in the commercial design flow, and T_{tool2} is that on the machine-learning-based design flow. $T^{*}_{eng.}$ is the time for fixing timing violations by an engineer, and the design flow repeats $\#Ite.^{*}$ times to meet the timing constraints. The acceleration rate between the commercial and the machine-learning-based method is shown by *Acc.* What is most significant is that our proposed method can achieve MAE less than 3 ps accurately, which can be regarded as having the same effect as the STA tool in practical industrial applications. We also verified the effect of the machine-learning-based method and the commercial method at the signoff stage, and the effort on fixing timing violations at the signoff stage is the same. Thus, in these industrial designs, we achieved more than 2x acceleration on timing closure and proved the proposed method. Note that the acceleration rate can be calculated by T_{tool1} $(T_{tool2} + T_{model})$ = 7x when only considering the runtime of the STA tool. Furthermore, our proposed method can be used as a "plug-in" for most STA tools, such as commercial STA tools or some new STA tools such as [14,27].

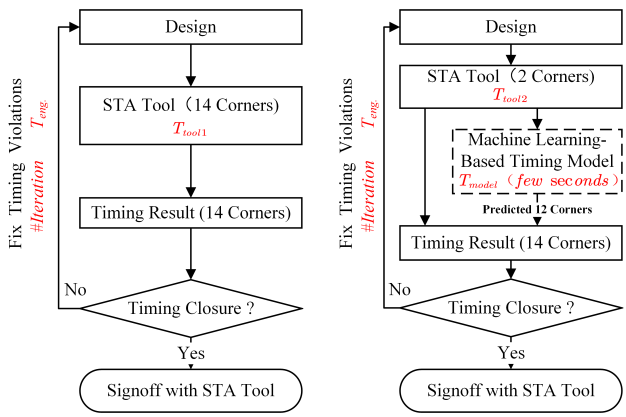


Figure 10. The commercial timing closure process (**left**) and machine-learning-based timing closure process (**right**).

Table 6. Runtime of commercial and machine-learning-based method in Industrial Application

Designs	Commercial				Machine Leraning-Based				Acc.
	T_{tool1}	$T^*_{eng.}$	$\#Ite.^*$	T_{total}	T_{tool2}	$T^*_{eng.}$	$\#Ite.^*$	T_{total}	
art1	0.59×14	4.5	15	191.4	0.59×2	4.5	15	85.2	2.24×
art2	0.88×14	6	20	366.4	0.88×2	6	20	155.2	2.36×

* is the typical value by a skilled engineer, and the value can vary by engineer. The unit of T is hours. The runtime of the model T_{model} is only a few seconds and can be ignored.

6. Conclusions

A traditional design flow typically undergoes numerous timing optimizations throughout the entire design flow to achieve timing closure. The long timing calculation process with static timing analysis (STA) tools has become the bottleneck of physical design efficiency. In this work, we proposed a dominant corner selection strategy (iterative increase strategy), which is used to quickly select dominant corner combinations to achieve high-accuracy prediction, thus accelerating the iterative timing closure process by accurate and fast timing prediction instead of querying a STA tool. Then, we proposed an application flow framework to integrate our multi-corner timing prediction into the industrial design flow. The experimental results show that, compared with the existing strategy, the prediction performance of the dominant corner combination selected by our iterative increase strategy is much better. The improvement of $LESS10$ is up to 15.0% in a design and can effectively drive the timing closure faster in industrial applications. In addition, we evaluated the prediction performance of different models, small-scale training data and incremental re-training. Generally speaking, our proposed timing prediction method has broad application prospects in physical chip design.

Author Contributions: Conceptualization, Z.Z., S.Z. and G.L.; methodology, Z.Z., S.Z. and C.F.; validation, G.L.; formal analysis, G.L. and C.F.; investigation, S.Z. and G.L.; resources, Z.Z., G.L., C.F., T.Y., A.H. and L.W.; data curation, S.Z., G.L., T.Y. and A.H.; writing—original draft preparation, G.L. and S.Z.; writing—review and editing, Z.Z., S.Z., G.L., C.F. and L.W.; visualization, G.L. and S.Z.; project administration, Z.Z., G.L., S.Z. and L.W.; All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Key Program of National Natural Science Foundation of China under Grant 62034005.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Public available dataset from ITC'99 benchmark circuit was analyzed in this study

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Chadha, R.; Bhasker, J. *Static Timing Analysis for Nanometer Designs: A Practical Approach*; Springer: Berlin/Heidelberg, Germany, 2009.
2. Kahng, A.B.; Mallappa, U.; Saul, L.; Tong, S. "Unobserved Corner" Prediction: Reducing Timing Analysis Effort for Faster Design Convergence in Advanced-Node Design. In Proceedings of the 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE), Florence, Italy, 25–29 March 2019; pp. 168–173.
3. e Silva, L.G.; Silveira, L.M.; Phillips, J.R. Efficient computation of the worst-delay corner. In Proceedings of the 2007 Design, Automation & Test in Europe Conference & Exhibition, Nice, France, 16–20 April 2007; pp. 1–6.
4. Onaissi, S.; Najm, F.N. A linear-time approach for static timing analysis covering all process corners. *IEEE Trans. -Comput.-Aided Des. Integr. Circuits Syst.* **2008**, *27*, 1291–1304. [CrossRef]
5. Orshansky, M.; Bandyopadhyay, A. Fast statistical timing analysis handling arbitrary delay correlations. In Proceedings of the ACM, San Diego, CA, USA, 7–11 June 2004; pp. 337–342.
6. Khandelwal, V.; Srivastava, A. A general framework for accurate statistical timing analysis considering correlations. In Proceedings of the Design Automation Conference, San Diego, CA, USA, 13–17 June 2005; pp. 89–94.

7. Nian, J.J.; Tsai, S.H.; Huang, C.Y. A unified Multi-Corner Multi-Mode static timing analysis engine. In Proceedings of the 2010 15th Asia and South Pacific Design Automation Conference (ASP-DAC), Taipei, Taiwan, 18–21 January 2010; pp. 669–674. doi: [CrossRef]
8. Onaissi, S.; Taraporevala, F.; Liu, J.; Najm, F. A fast approach for static timing analysis covering all PVT corners. In Proceedings of the 2011 48th ACM/EDAC/IEEE Design Automation Conference (DAC), San Diego, California, USA, 5–10 June 2011; pp. 777–782.
9. Bian, S.; Hiromoto, M.; Shintani, M.; Sato, T. LSTA: Learning-based static timing analysis for high-dimensional correlated on-chip variations. In Proceedings of the 2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC), Austin, TX, USA, 18–22 June 2017; pp. 1–6.
10. Kahng, A.B.; Luo, M.; Nath, S. SI for free: machine learning of interconnect coupling delay and transition effects. In Proceedings of the 2015 ACM/IEEE International Workshop on System Level Interconnect Prediction (SLIP), San Francisco, CA, USA, 6 June 2015; pp. 1–8.
11. Han, S.S.; Kahng, A.B.; Nath, S.; Vydyanathan, A.S. A deep learning methodology to proliferate golden signoff timing. In Proceedings of the 2014 Design, Automation & Test in Europe Conference & Exhibition (DATE), Dresden, Germany, 24–28 March 2014; pp. 1–6.
12. Kahng, A.B.; Kang, S.; Lee, H.; Nath, S.; Wadhwani, J. Learning-based approximation of interconnect delay and slew in signoff timing tools. In Proceedings of the 2013 ACM/IEEE International Workshop on System Level Interconnect Prediction (SLIP), Austin, TX, USA, 2 June 2013; pp. 1–8.
13. Chan, W.T.J.; Chung, K.Y.; Kahng, A.B.; Macdonald, N.D.; Nath, S. Learning-based prediction of embedded memory timing failures during initial floorplan design. In Proceedings of the Asia & South Pacific Design Automation Conference, Macao, Macao, 25–28 January 2016; pp. 178–185.
14. Guo, Z.; Huang, T.W.; Lin, Y. Gpu-accelerated static timing analysis. In Proceedings of the 39th International Conference on Computer-Aided Design, San Diego, CA, USA, 2–5 November 2020; pp. 1–9.
15. Deng, W.; Okada, K.; Matsuzawa, A. A feedback class-C VCO with robust startup condition over PVT variations and enhanced oscillation swing. In Proceedings of the ESSCIRC (ESSCIRC), Helsinki, Finland, 12–16 September 2011; pp. 499–502.
16. Chang, K.j.; Chang, L.f.; Mathews, R.G.; Walker, M.G. Method and System for Extraction of Parasitic Interconnect Impedance Including Inductance. US Patent 6,643,831, 2003.
17. Marquaridt, D.W. Generalized inverses, ridge regression, biased linear estimation, and nonlinear estimation. *Technometrics* **1970**, *12*, 591–612. [CrossRef]
18. Gardner, M.W.; Dorling, S. Artificial neural networks (the multilayer perceptron)—A review of applications in the atmospheric sciences. *Atmos. Environ.* **1998**, *32*, 2627–2636. [CrossRef]
19. Breiman, L. Random forests. *Mach. Learn.* **2001**, *45*, 5–32. [CrossRef]
20. Zhang, S.Z.; Zhao, Z.Y.; Feng, C.C.; Wang, L. A Machine Learning Framework with Feature Selection for Floorplan Acceleration in IC Physical Design. *J. Comput. Sci. Technol.* **2020**, *35*, 468–474. [CrossRef]
21. Benesty, J.; Chen, J.; Huang, Y.; Cohen, I. Pearson correlation coefficient. In *Noise Reduction in Speech Processing*; Springer: Berlin/Heidelberg, Germany, 2009; pp. 1–4.
22. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. Scikit-learn: Machine learning in Python. *J. Mach. Learn. Res.* **2011**, *12*, 2825–2830.
23. Huang, T.W.; Wong, M.D. OpenTimer: A high-performance timing analysis tool. In Proceedings of the 2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), Austin, TX, USA, 2–6 November 2015; pp. 895–902.
24. Davidson, S. Characteristics of the ITC'99 benchmark circuits. In Proceedings of the IEEE International Test Synthesis Workshop (ITSW), Atlantic City Nj, USA, 28–30 September 1999.
25. Ranjan, G.; Verma, A.K.; Radhika, S. K-nearest neighbors and grid search cv based real time fault monitoring system for industries. In Proceedings of the 2019 IEEE 5th International Conference for Convergence in Technology (I2CT), Bombay, India, 29–31 March 2019; pp. 1–5.
26. Jung, Y.; Hu, J. AK-fold averaging cross-validation procedure. *J. Nonparametr. Stat.* **2015**, *27*, 167–179. [CrossRef] [PubMed]
27. Huang, T.W.; Guo, G.; Lin, C.X.; Wong, M.D. OpenTimer v2: A new parallel incremental timing analysis engine. *IEEE Trans. -Comput.-Aided Des. Integr. Circuits Syst.* **2020**, *40*, 776–789. [CrossRef]